Autor:
Ana Isabel Vegas

ibertech

end2end dedication

CURSO

Microservicios con Python y Flask

www.ibertech.es

ibertech

end2end dedication

Tema 1	Introducción a Python y Flask en la arquitectura de microservicios
Tema 2	Creación de API RestFul con Python y Flask
Tema 3	Comunicación entre microservicios
Tema 4	Seguridad y autenticación
Tema 5	Despliegue de microservicios con Docker y kubernetes
Tema 6	Monitorización y Gestión de microservicios con Python

ibertech

end2end dedication

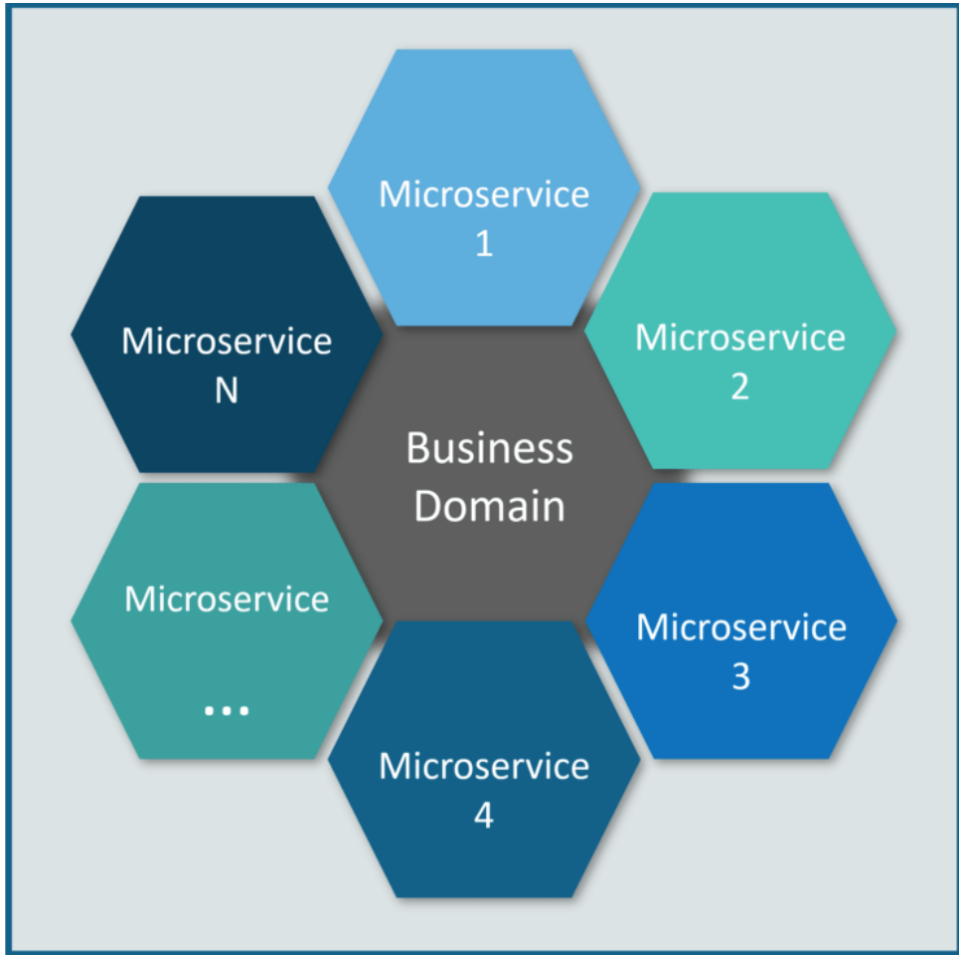
Tema 1

Introducción a Python y Flask en la arquitectura de microservicios

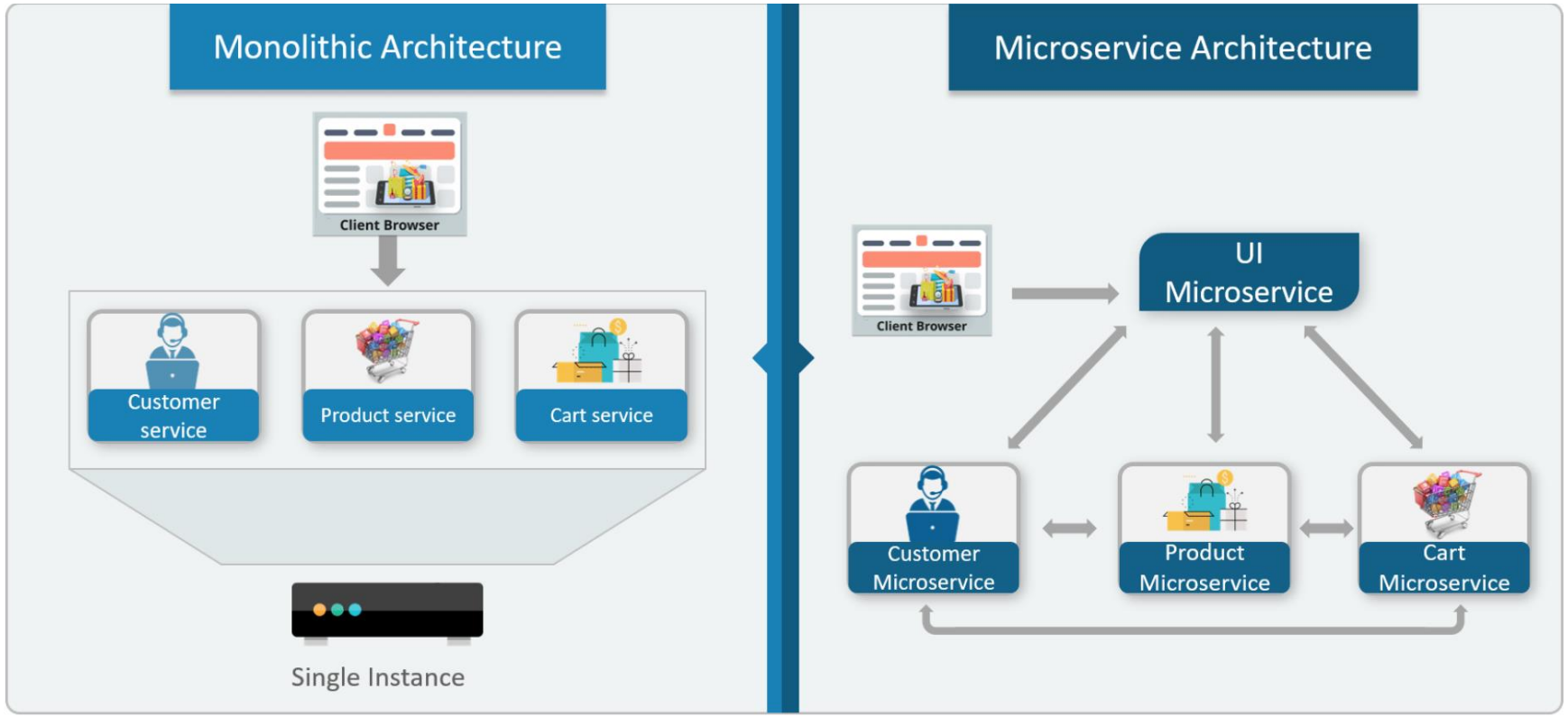
Qué es un microservicio?

- Según *Martin Fowler* y *James Lewis* explican en su artículo *Microservices*, los **microservicios** se definen como un estilo arquitectural, es decir, una forma de desarrollar una aplicación, basada en un conjunto de pequeños servicios, cada uno de ellos ejecutándose de forma autónoma y comunicándose entre si mediante mecanismos livianos, generalmente a través de peticiones **REST** sobre HTTP por medio de sus **APIs**.

Qué es un microservicio?



Arquitectura monolítica vs Arquitectura microservicios



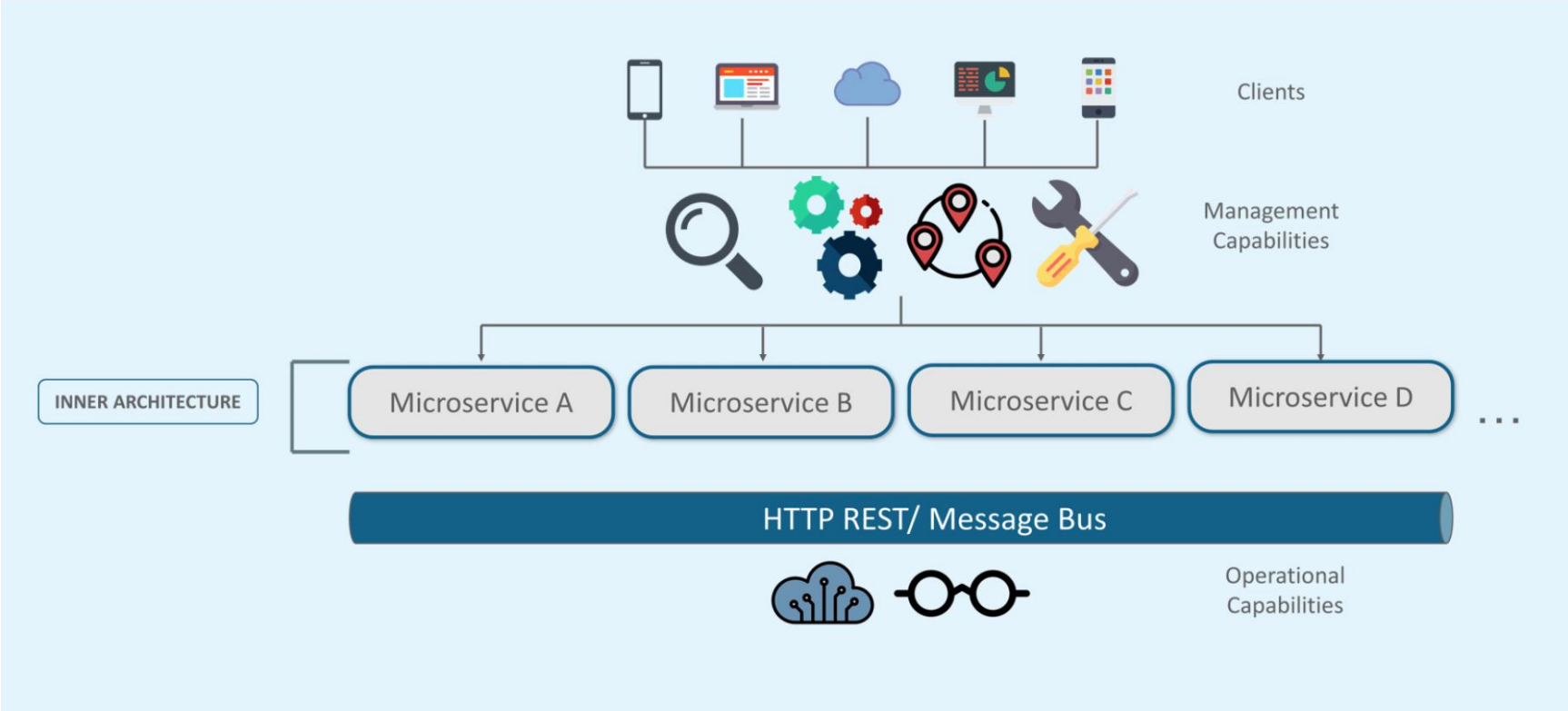
Tendencia en el desarrollo

- La tendencia es que las aplicaciones sean diseñadas con un ***enfoque orientado a microservicios***, construyendo múltiples servicios que colaboran entre si, en lugar del ***enfoque monolítico***, donde se construye y despliega una única aplicación que contenga todas las funcionalidades.

Características de los Microservicios

- Deben ser auto-contenidos, de tal forma que incluyen todo lo necesario para prestar su servicio
- Servicios pequeños, lo que facilita el mantenimiento. Ej: Personas, Productos, Posición Global, etc
- Principio de responsabilidad única: cada microservicio hará una única cosa, pero la hará bien
- Políglotas: una arquitectura basada en microservicios facilita la integración entre diferentes tecnologías (lenguajes de programación, BBDD…etc)
- Despliegues unitarios: los microservicios pueden ser desplegados por separado, lo que garantiza que cada despliegue de un microservicio no implica un despliegue de toda la plataforma. Tienen la posibilidad de incorporar un servidor web embebido como Tomcat o Jetty
- Escalado eficiente: una arquitectura basada en microservicios permite un escalado elástico horizontal, pudiendo crear tantas instancias de un microservicio como sea necesario.

Arquitectura microservicios



Arquitectura microservicios

- Diferentes clientes de diferentes dispositivos intentan usar diferentes servicios como búsqueda, creación, configuración y otras capacidades de administración
- Todos los servicios se separan según sus dominios y funcionalidades y se asignan a microservicios individuales.
- Estos microservicios tienen su propio balanceador de carga y entorno de ejecución para ejecutar sus funcionalidades y al mismo tiempo captura datos en sus propias bases de datos.
- Todos los microservicios se comunican entre sí a través de un servidor sin estado que es REST o Message Bus.
- Los microservicios conocen su ruta de comunicación con la ayuda de Service Discovery y realizan capacidades operativas tales como automatización, monitoreo
- Luego, todas las funcionalidades realizadas por los microservicios se comunican a los clientes a través de la puerta de enlace API.
- Todos los puntos internos están conectados desde la puerta de enlace API. Por lo tanto, cualquiera que se conecte a la puerta de enlace API se conecta automáticamente al sistema completo

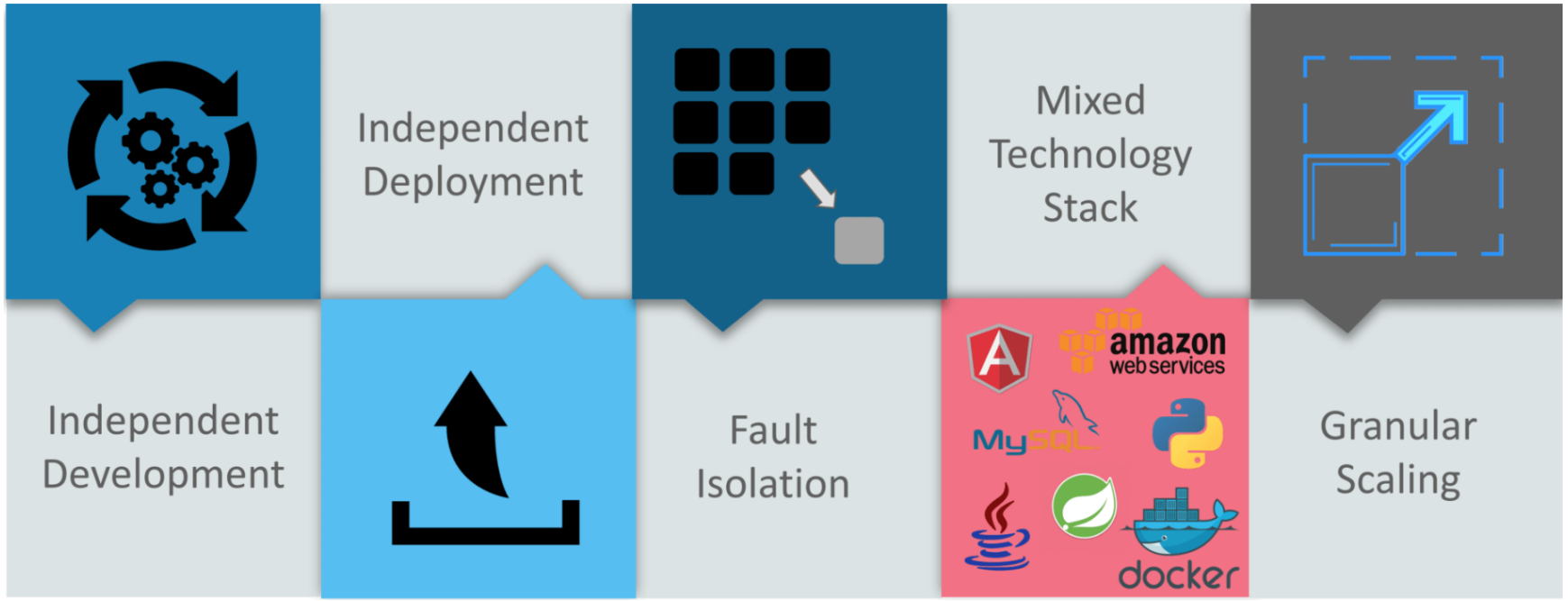
Características de los microservicios



Características de los microservicios

- Desacoplamiento: los servicios dentro de un sistema se desacoplan en gran medida. Por lo tanto, la aplicación en su conjunto se puede construir, modificar y escalar fácilmente.
- Componentes: los microservicios se tratan como componentes independientes que se pueden reemplazar y actualizar fácilmente.
- Capacidades empresariales: los microservicios son muy simples y se centran en una sola capacidad
- Autonomía: los desarrolladores y los equipos pueden trabajar de forma independiente, lo que aumenta la velocidad.
- Entrega continua: permite lanzamientos frecuentes de software, a través de la automatización sistemática de la creación, prueba y aprobación del software.
- Responsabilidad: Los microservicios no se centran en aplicaciones como proyectos. En cambio, tratan las aplicaciones como productos de los que son responsables.
- Gobernanza descentralizada: el enfoque está en usar la herramienta adecuada para el trabajo correcto. Eso significa que no hay un patrón estandarizado o ningún patrón tecnológico. Los desarrolladores tienen la libertad de elegir las mejores herramientas útiles para resolver sus problemas
- Agilidad - Los microservicios apoyan el desarrollo ágil. Cualquier nueva característica puede ser desarrollada rápidamente y descartada nuevamente

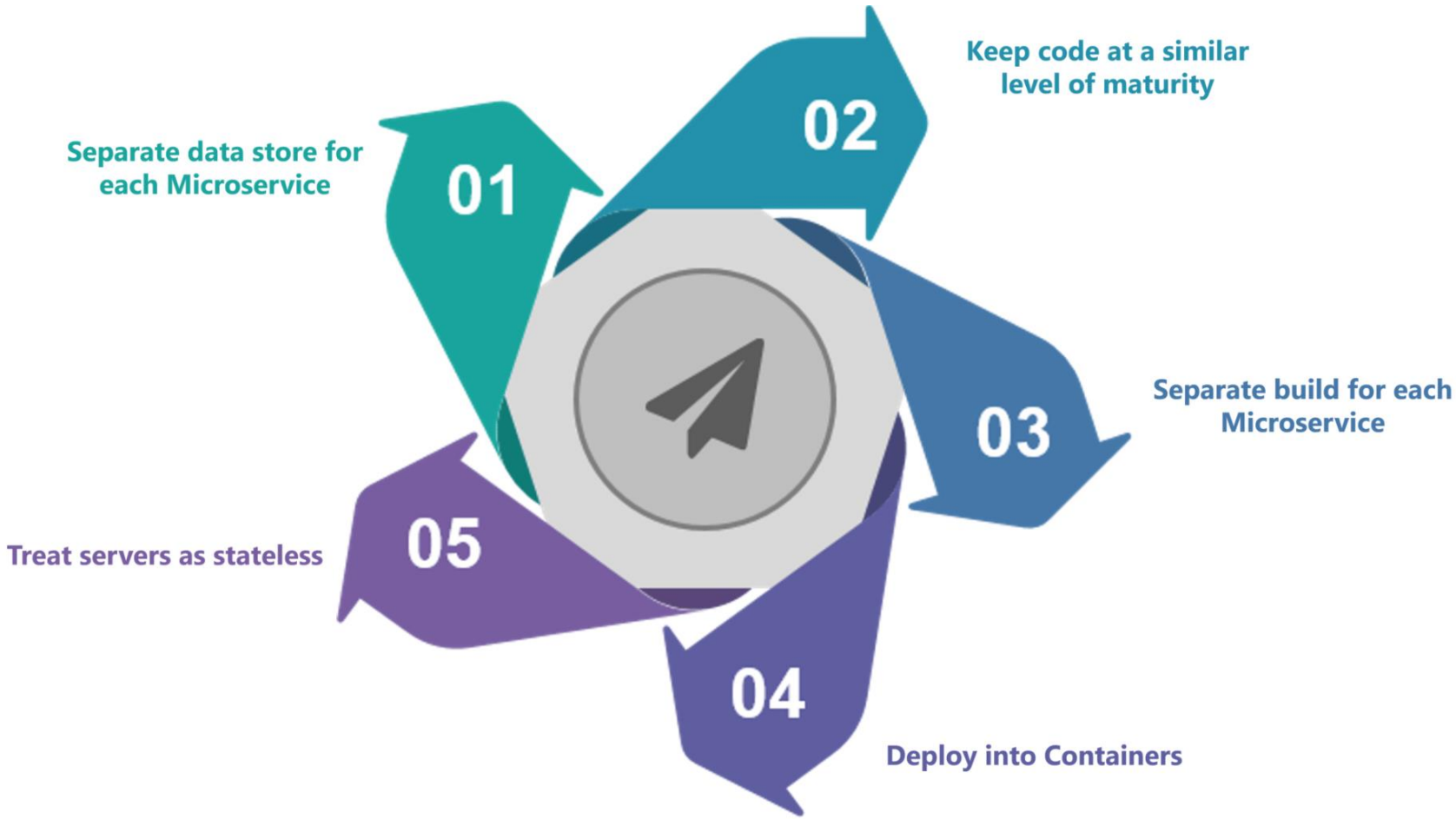
Ventajas de los microservicios



Ventajas de los microservicios

- Desarrollo independiente: todos los microservicios se pueden desarrollar fácilmente según su funcionalidad individual
- Implementación independiente: en función de sus servicios, se pueden implementar individualmente en cualquier aplicación
- Aislamiento de fallos: incluso si un servicio de la aplicación no funciona, el sistema continúa funcionando
- Pila de tecnología mixta: se pueden utilizar diferentes lenguajes y tecnologías para crear diferentes servicios de la misma aplicación
- Escalado granular: los componentes individuales pueden escalarse según la necesidad, no es necesario escalar todos los componentes

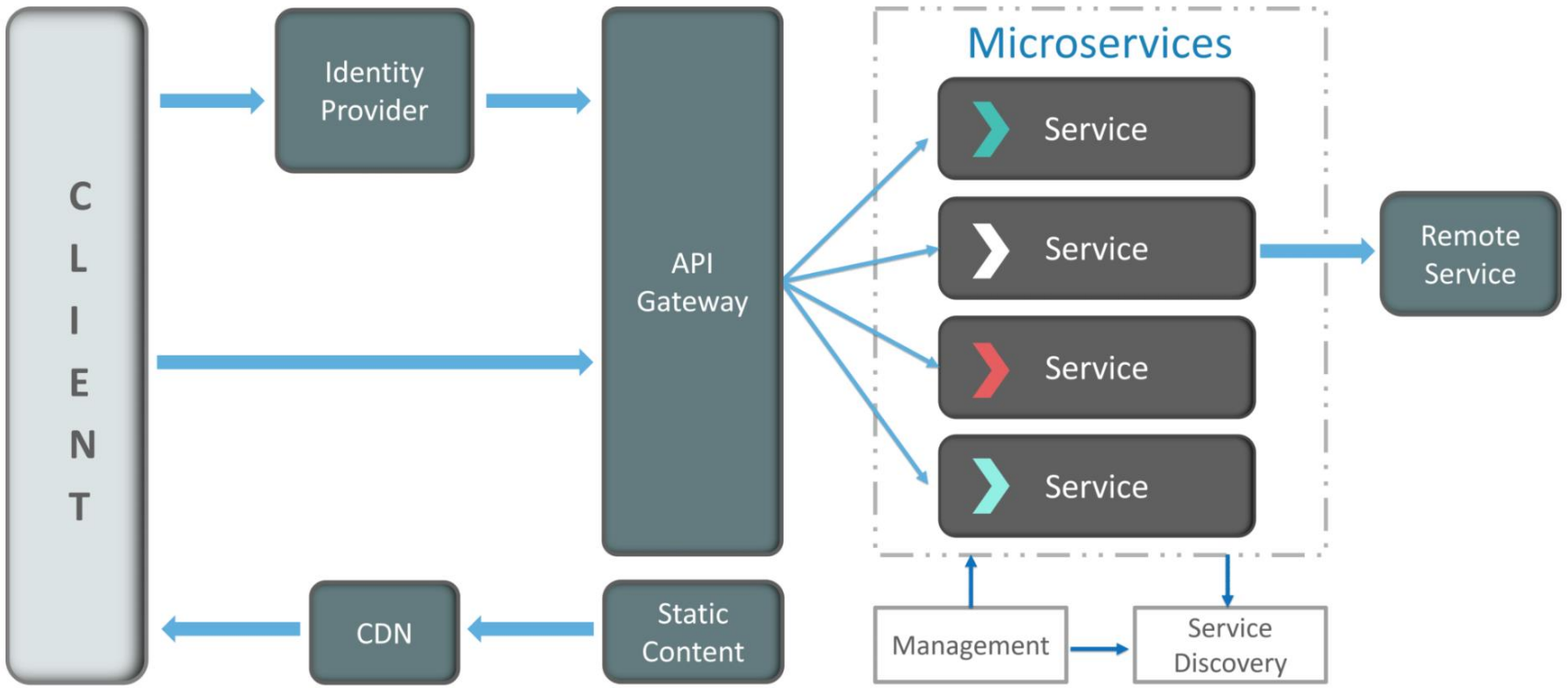
Buenas prácticas de diseño



Quien los utiliza?



Componentes de la arquitectura



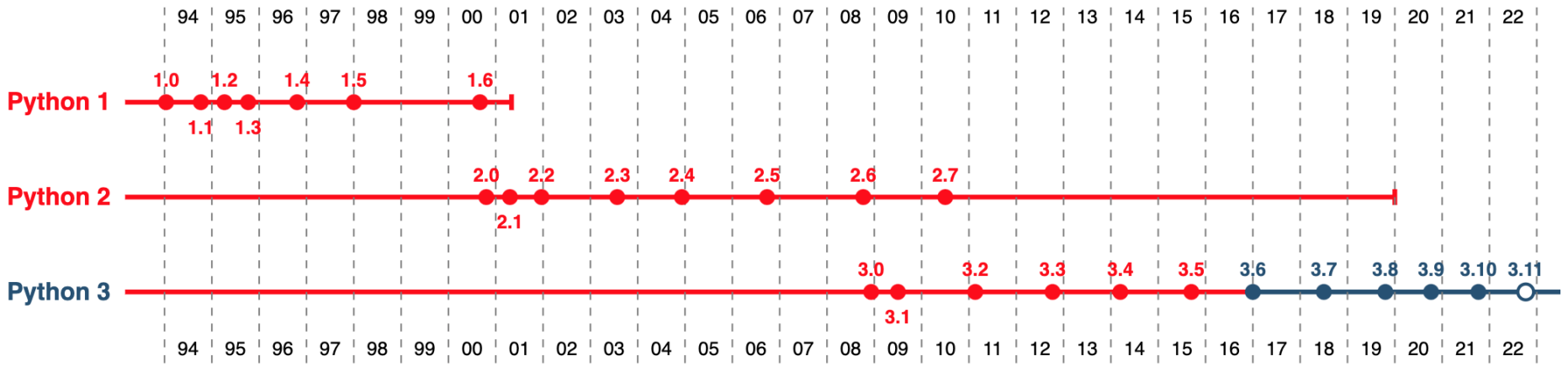
Arquitectura microservicios

- Diferentes clientes de diferentes dispositivos intentan usar diferentes servicios como búsqueda, creación, configuración y otras capacidades de administración
- Todos los servicios se separan según sus dominios y funcionalidades y se asignan a microservicios individuales.
- Estos microservicios tienen su propio balanceador de carga y entorno de ejecución para ejecutar sus funcionalidades y al mismo tiempo captura datos en sus propias bases de datos.
- Todos los microservicios se comunican entre sí a través de un servidor sin estado que es REST o Message Bus.
- Los microservicios conocen su ruta de comunicación con la ayuda de Service Discovery y realizan capacidades operativas tales como automatización, monitoreo
- Luego, todas las funcionalidades realizadas por los microservicios se comunican a los clientes a través de la puerta de enlace API.
- Todos los puntos internos están conectados desde la puerta de enlace API. Por lo tanto, cualquiera que se conecte a la puerta de enlace API se conecta automáticamente al sistema completo

Qué es Python?

- Python es un lenguaje de programación que fue creado a finales de los años 80 por el holandés Guido van Rossum, fan del grupo humorístico Monty Python, de ahí el nombre que le puso al lenguaje de programación.

Versiones de Python



Características de Python

- **Simpleza**: es un lenguaje muy sencillo, de ahí el gran exito de Python.
- **Sintaxis clara**: es obligatoria la indentación
- **Propósito general**: se pueden crear todo tipo de programas
- **Lenguaje interpretado**: no es necesaria compilación, el interprete de Python lo ejecuta.
- **Lenguaje de alto nivel**: no hay que preocuparse del manejo de memoria y otros aspectos de bajo nivel
- **Lenguaje orientado a objetos**: Favorece la reutilización de código
- **Open source**: es un lenguaje de programación gratuito
- **Extensas librerías**: incluyen muchas funcionalidades
- **Incrustable**: es posible añadir programas Python a programas en C y C++.

Ventajas

- **Legible**: sintaxis intuitiva y estricta
- **Productivo**: ahorra mucho código
- **Portable**: para todo sistema operativo
- **Recargado**: viene con muchas librerías por defecto

Instalación en Windows

- Desde la página www.python.org/download se descarga y se instala.
- El sitio reconocerá el sistema operativo y te dará las opciones para descargar.



- Ejecuta el archivo que descargaste y sigue los pasos de instalación.

Comprobar versión instalada

- Desde la terminal de comandos:

```
anabel@MacBook-Air-de-Anabel ~ % python3 --version
Python 3.14.3
```

Qué es Flask?

- Flask es un microframework web para Python que permite crear aplicaciones web y APIs HTTP de forma sencilla, sin obligarte a utilizar una estructura de proyecto ni un conjunto de herramientas determinado.
- Está construido sobre Werkzeug (servidor y utilidades HTTP) y Jinja (motor de plantillas).
- Su idea fundamental es: proporcionarte lo necesario para recibir peticiones HTTP y devolver respuestas, dejando que tú decidas el resto.
- Web oficial: https://flask.palletsprojects.com/en/stable/

Ventajas de Flask

- **Curva de aprendizaje baja**. Si conoces Python, puedes crear tu primera API en minutos.
- **Poca complejidad inicial**. No necesitas aprender un sistema de configuración extenso ni una arquitectura impuesta por el framework.
- **Flexibilidad**. Puedes elegir tu ORM, base de datos, sistema de autenticación, cliente HTTP y demás dependencias.
- **Ideal para APIs pequeñas**. Es sencillo crear servicios que exponen unos pocos endpoints REST.
- **Ecosistema amplio**. Hay extensiones para autenticación, bases de datos, migraciones, validación y muchas otras necesidades.

Flask vs otros frameworks

- **Flask**
  - Minimalista
  - Ideal para: microservicios sencillos y APIs REST pequeñas.
  - Ofrece libertad para decidir cómo organizar el proyecto y qué librerías utilizar.
- **Django**
  - Completo
  - Ideal para: aplicaciones web grandes y sistemas de gestión.
  - Incluye ORM, administración, autenticación y muchas otras funcionalidades integradas. Puede resultar excesivo para un microservicio pequeño.
- **FastAPI**
  - Orientado a APIs
  - Ideal para: APIs modernas con validación y documentación automática.
  - Utiliza anotaciones de tipos, Pydantic y soporte para operaciones asíncronas. Genera documentación OpenAPI automáticamente.

ibertech

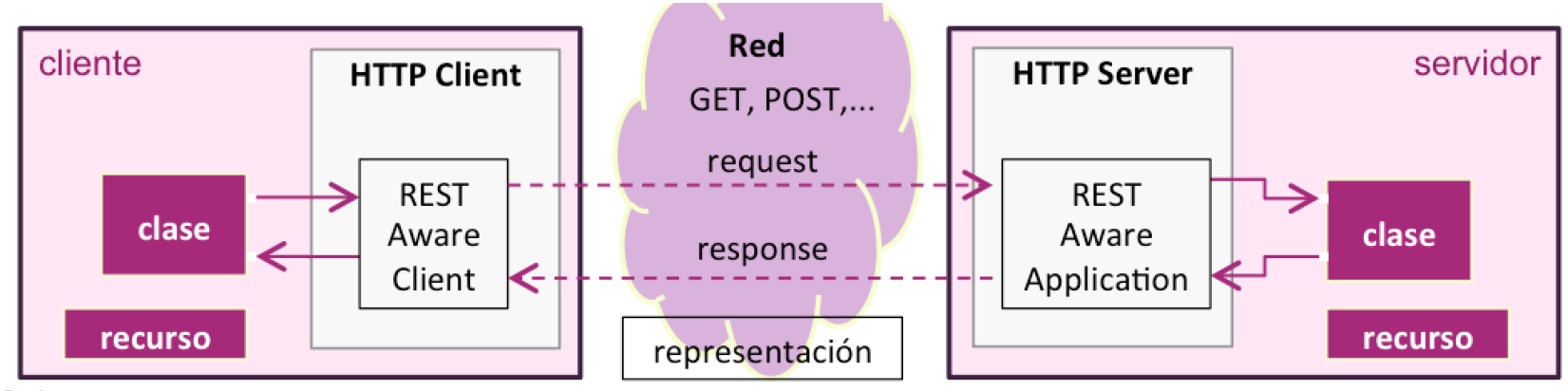
end2end dedication

Tema 2

Creación de API RestFul con Python y Flask

Creando RESTfull Web Service

- REST (Representational State Transfer) es un estilo de arquitectura para sistemas distribuidos, desarrollada por la W3C, junto con el protocolo HTTP.
- Las arquitecturas REST tienen clientes y servidores.
- El cliente realiza un envío (request) al servidor, el cual lo procesa y retorna una respuesta al cliente.
- Las peticiones y respuestas son construidas alrededor de representaciones de recursos. Recurso es una entidad, y representación es cómo se formatea.



Creando RESTfull Web Service

- Una API del tipo RESTful, o RESTful Web Service, es una API web implementada con HTTP y los principios REST, con los siguientes aspectos:
  - Una URI base del servicio.
  - Un formato de mensajes, por ejemplo JSON o XML.
  - Un conjunto de operaciones, que utilizan los métodos HTTP (GET, PUT, POST o DELETE).

- La API debe manejar hipertextos.
- A diferencia de los Web Services basados en SOAP, no hay un estándar comúnmente aceptado para los RESTful. Esto es porque REST es una arquitectura, mientras que SOAP es un protocolo.
- Esta desventaja se compensa con la simplicidad de su utilización y el bajo consumo de recursos durante el binding. Esto es especialmente útil en aplicaciones para dispositivos móviles

Creando RESTfull Web Service

- Con REST, los métodos HTTP se asocian a tipos de operaciones sobre recursos. El uso comúnmente aceptado es el siguiente:
    - GET: Para recuperar la representación de un recurso. Es idempotente, es decir, si se invoca múltiples veces, retorna el mismo resultado.
    - POST: Para crear un recurso, o para actualizarlo. También, por las características del método, se utiliza para envíos grandes, o para evitar limitaciones de los otros métodos.
    - PUT: Para actualizar un recurso, ya que POST no es idempotente.
    - DELETE: Para eliminar un recurso.
    - OPTIONS: Se puede utilizar para hacer un "ping" del servicio, es decir, verificar su disponibilidad.
    - HEAD: Para buscar un recurso o consultar estado. Similar a GET

Definición y creación de endpoints RESTful

- Un **endpoint** es una combinación de una URL y un método HTTP que permite acceder a una funcionalidad de la API.

Método	Endpoint	Función
GET	`/productos`	Obtener todos los productos
GET	`/productos/1`	Obtener un producto
POST	`/productos`	Crear un producto
PUT	`/productos/1`	Reemplazar un producto
DELETE	`/productos/1`	Eliminar un producto

Validaciones con Marshmallow

- **Marshmallow** es una biblioteca de Python para validar, serializar y deserializar datos. Es especialmente útil cuando desarrollas APIs REST que reciben y devuelven JSON.
- Si quieres dar de alta un nuevo producto:

```
{
  "descripcion": "Teclado mecánico",
  "precio": 59.90
}
```

- ¿Cómo compruebas que la descripción es una cadena, que el precio es numérico y que no es negativo?
- Marshmallow permite definir las reglas una sola vez en un **esquema** (*schema*) y reutilizarlas.

Validaciones con Marshmallow

- Para realizar las validaciones con Marshmallow llevamos a cabo estas 3 operaciones:

Operación	Método Marshmallow	Finalidad
Serializar	`dump()`	Preparar datos Python para devolverlos al cliente
Deserializar	`load()`	Validar y transformar datos recibidos
Consultar errores	`ValidationError.messages`	Obtener los errores de validación

Esquemas (Schemas)

- Un esquema define cómo debe ser un objeto: qué campos tiene, qué tipos de datos admite y qué restricciones debe cumplir.
- Por ejemplo, un producto tiene:
  - id: entero positivo.
  - descripcion: texto obligatorio.
  - precio: número decimal no negativo.

```python
from marshmallow import Schema, fields, validate

class ProductoSchema(Schema):
    id = fields.Int(
        required=True,
        validate=validate.Range(min=1)
    )

    descripcion = fields.Str(
        required=True,
        validate=validate.Length(min=1)
    )

    precio = fields.Float(
        required=True,
        validate=validate.Range(min=0)
    )
```

Serialización

- La **serialización** convierte los datos de Python en una representación que se puede enviar a otro sistema, normalmente JSON.
- dump() genera una estructura de datos Python preparada para su representación externa. En Flask, podemos devolverla con jsonify().

```python
producto = {
    "id": 1,
    "descripcion": "Teclado mecánico",
    "precio": 59.90
}
```

```python
schema = ProductoSchema()

resultado = schema.dump(producto)
```

Deserialización

- La **deserialización** realiza el proceso inverso: recibe datos externos y los transforma en datos Python validados.
- Si el esquema admite la conversión, resultado contendrá un diccionario con los tipos de datos normalizados, como una cadena para la descripción y un número para el precio.
- Si los datos no cumplen las reglas, load() lanza una excepción ValidationError con los errores encontrados.

```
datos = {
    "descripcion": "Teclado mecánico",
    "precio": "59.90"
}

resultado = ProductoEntradaSchema().load(datos)
```

Manejo de errores y excepciones

- Códigos HTTP habituales:

Código	Significado	Ejemplo en nuestro microservicio
`200 OK`	Operación correcta	Consultar un producto
`201 Created`	Recurso creado	Crear un producto
`204 No Content`	Operación correcta sin respuesta	Eliminar un producto
`400 Bad Request`	Petición incorrecta	JSON inválido o precio negativo
`404 Not Found`	Recurso inexistente	Consultar el producto 99
`405 Method Not Allowed`	Método no permitido	Enviar DELETE a una ruta que solo admite GET
`415 Unsupported Media Type`	Tipo de contenido no admitido	Enviar un cuerpo que no es JSON
`500 Internal Server Error`	Error inesperado del servidor	Excepción no controlada

Manejadores de errores

- Flask permite registrar funciones que se ejecutan cuando se produce un error determinado.
- Así podemos evitar que Flask devuelva páginas HTML de error y proporcionar una respuesta JSON uniforme.

```python
@app.errorhandler(404)
def no_encontrado(error):
    return {
        "error": {
            "codigo": "RECURSO_NO_ENCONTRADO",
            "mensaje": "El recurso solicitado no existe"
        }
    }, 404
```

Swagger

- Swagger es el conjunto de herramientas asociado al ecosistema OpenAPI.
- En una aplicación Flask, podemos generar la documentación a partir del código, de las anotaciones o de una especificación OpenAPI.
- Para nuestro curso vamos a empezar con Flask-RESTPlus, que permite definir recursos REST, modelos y documentación mediante decoradores.
- Es importante conocer una particularidad: Flask-RESTPlus es una biblioteca antigua y su desarrollo original está discontinuado. Para un proyecto nuevo suele ser preferible Flask-RESTX, su continuación comunitaria, o una solución basada directamente en OpenAPI.

ibertech

end2end dedication

Tema 3

Comunicación entre microservicios

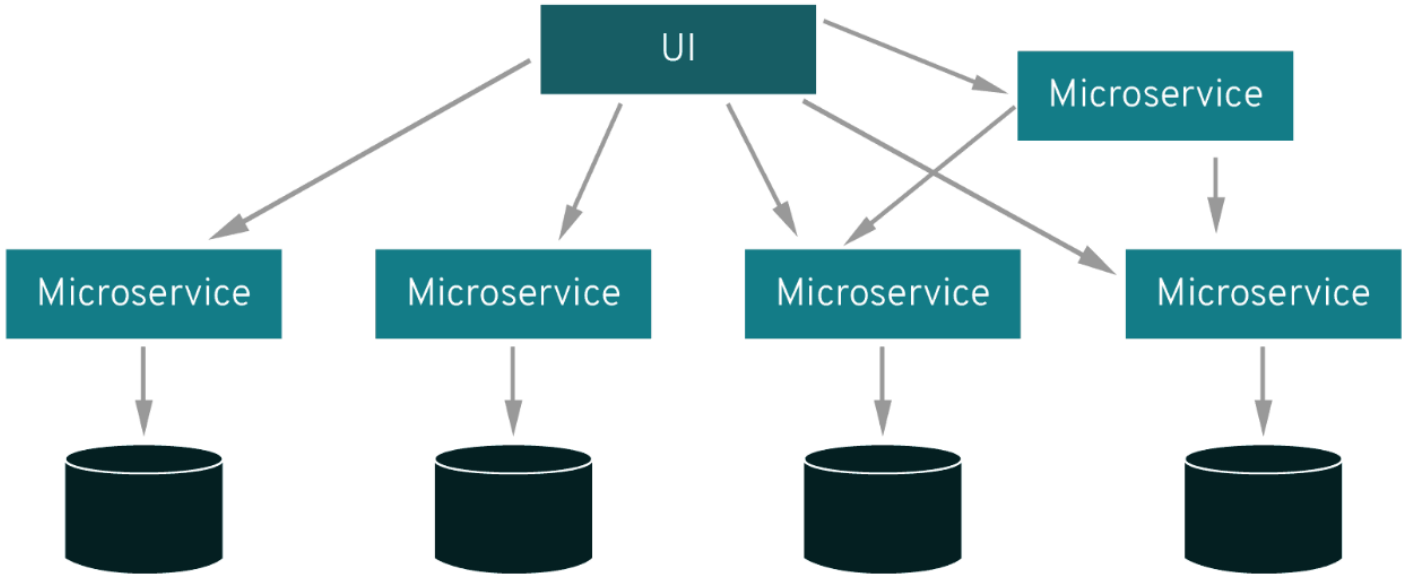
Comunicación entre microservicios

- Los microservicios se crean de forma independiente y se comunican entre sí. Además, si se produce un error individual, este no provoca una interrupción de toda la aplicación.

- La comunicación se lleva a cabo a través de peticiones Rest.

- Dos formas de implementar la comunicacion:
  - Síncrona: Comunicación HTTP
  - Asíncrona con eventos

Comunicación entre microservicios



Arquitectura basada en eventos

- Una arquitectura basada en eventos, conocida como *Event-Driven Architecture* (EDA), es una forma de diseñar aplicaciones en la que los servicios se comunican mediante **eventos que indican que algo ha sucedido**.

- Un evento no es lo mismo que una petición. La petición solicita una acción; el evento comunica que algo ha sucedido.

- La diferencia fundamental es que, con HTTP, un servicio llama directamente a otro. Con eventos, un servicio publica un mensaje y otros servicios pueden reaccionar sin que el emisor tenga que conocerlos individualmente.

- En Python puedes utilizar distintas tecnologías:

  - **RabbitMQ:** permite trabajar con colas, confirmaciones y distintos patrones de enrutamiento.
  - **Apache Kafka:** está orientado a flujos de eventos y registros distribuidos.
  - **Redis Streams:** permite trabajar con flujos de mensajes y grupos de consumidores.

Comunicación asíncrona con eventos

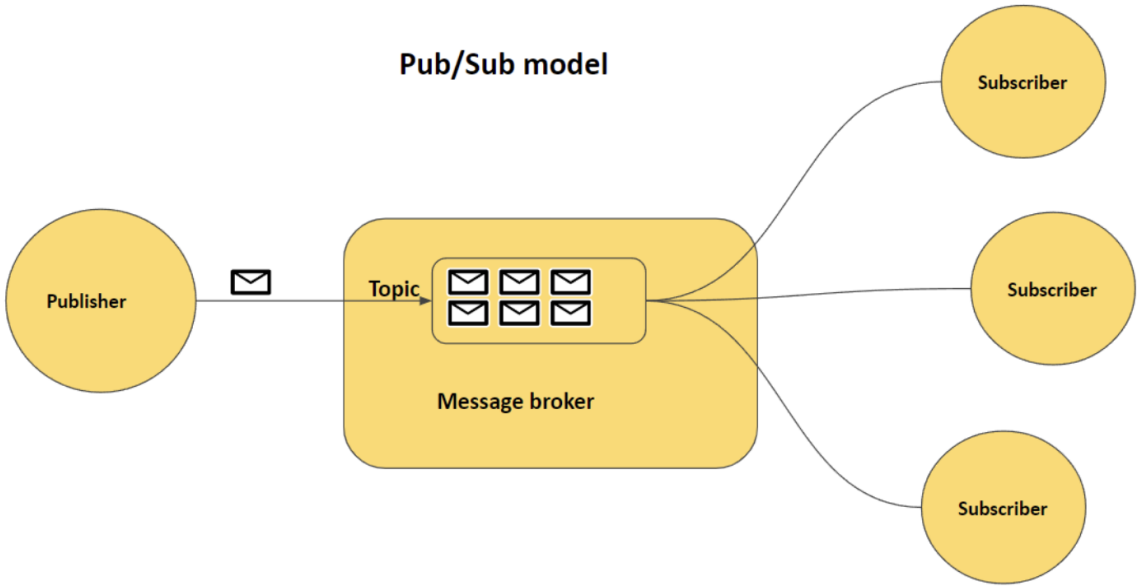
- Ventajas principales:
  - Desacoplamiento: Los servicios no necesitan conocerse entre sí, solo el canal y el formato del mensaje.
  - Escalabilidad y resiliencia: Los mensajes pueden ser procesados por múltiples instancias y recuperarse ante fallos.
  - Asincronía real: El productor no espera la respuesta del consumidor, permitiendo procesar grandes volúmenes de eventos sin bloqueo

Patrón Point-to-Point (punto a punto)

- El patrón Point-to-Point se utiliza cuando quieres enviar una tarea a una cola para que la procese un consumidor.
- Si hay diez tareas en la cola, se puede distribuirlas entre diferentes consumidores.
- Cada mensaje se entrega a **un solo consumidor** dentro de ese grupo de trabajadores, aunque pueden existir reintentos si un procesamiento falla.
- Este patrón es adecuado para:
  - Generar facturas.
  - Procesar pedidos.
  - Ejecutar trabajos en segundo plano.
  - Enviar tareas de larga duración a trabajadores.

Patrón Pub/Sub (publicación/suscripción)

- Un servicio publica un evento y **varios suscriptores pueden recibirlo**, cada uno para realizar una acción diferente.



Point-to-Point vs Pub/Sub

Característica	Point-to-Point	Pub/Sub
Objetivo	Ejecutar una tarea	Difundir un evento
Destino	Una cola	Un exchange que distribuye a varias colas
Consumidores	Compiten por los mensajes de la cola	Cada suscriptor tiene su propia cola
Ejemplo	Generar una factura	Notificar que se ha creado un pedido
Ventaja principal	Reparto del trabajo	Desacoplamiento entre servicios

Implementación de Celery para tareas distribuidas y asincrónicas en Python:

- Imagina que creas un pedido y, además de guardarlo, tienes que realizar varias operaciones:
    1. Consultar el producto.
    2. Crear el pedido.
    3. Enviar un correo de confirmación.
    4. Generar una factura.
    5. Registrar información para estadísticas.

- Si realizas todas estas operaciones dentro de la petición HTTP, el cliente tendrá que esperar hasta que terminen.
- Con Celery puedes delegar las tareas que no necesitan completarse antes de responder al cliente.
- La idea principal es que **la petición HTTP no tiene que esperar a que terminen todas las tareas secundarias**.

Celery

- Celery es una herramienta de Python que permite ejecutar tareas fuera del proceso principal de una aplicación.
- Sus componentes principales son:
  - Aplicación Flask: recibe las peticiones HTTP.
  - Productor de tareas: normalmente, el propio código Flask que solicita ejecutar una tarea.
  - Broker: transporta los mensajes que contienen las tareas pendientes.
  - Worker: proceso independiente que recoge y ejecuta las tareas.
  - Backend de resultados (opcional): almacena los estados y resultados de las tareas para poder consultarlos posteriormente.

- Celery permite ejecutar tareas en segundo plano y distribuirlas entre varios trabajadores, incluso en máquinas diferentes.

Redis

- Redis es un almacén de datos en memoria que también puede utilizarse como broker para Celery.
- Cuando Flask quiere ejecutar una tarea:
    1. Celery publica un mensaje en Redis.
    2. El worker de Celery obtiene ese mensaje.
    3. El worker ejecuta la función Python correspondiente.
    4. Si se configura un backend de resultados, el estado o resultado se puede consultar posteriormente.

- Redis no ejecuta el código Python de la tarea: **el worker es quien ejecuta la función; Redis ayuda a transportar el mensaje**.

Patrones de mensajería en Python

- Event Sourcing
- CQRS (Command Query Responsibility Segregation)

Patrón Event Sourcing

- Event Sourcing es un patrón de arquitectura en el que, en lugar de guardar únicamente el estado actual de un objeto, se guarda una secuencia de eventos que representan todos los cambios que ha experimentado.
- Por ejemplo, en tu microservicio de pedidos, podríamos registrar estos eventos:
    - PedidoCreado
    - CantidadActualizada
    - PedidoCancelado
- A partir de esos eventos podemos reconstruir el estado del pedido.
- Ventajas y desventajas
    - Ventajas: permite auditar cambios, reconstruir estados y conocer el historial completo de un pedido.
    - Desventajas: añade complejidad, requiere gestionar la evolución de los eventos y puede necesitar mecanismos para reconstruir estados de forma eficiente.

Patron CQRS

- CQRS son las siglas de Command Query Responsibility Segregation, que en español significa separación de responsabilidades entre comandos y consultas.
- La idea es dividir las operaciones de una aplicación en dos grupos:
    - Commands (comandos): modifican los datos. Por ejemplo, crear un pedido, cambiar su cantidad o cancelarlo.
    - Queries (consultas): leen los datos sin modificarlos. Por ejemplo, consultar un pedido por su ID o listar los pedidos.
- En una aplicación sencilla, ambas operaciones pueden utilizar la misma base de datos. CQRS permite separarlas y, cuando es necesario, utilizar modelos de datos diferentes para cada una.

Operación	Tipo CQRS	Función
Crear un pedido	Command	Registra un pedido nuevo.
Cancelar un pedido	Command	Cambia el estado del pedido.
Consultar un pedido	Query	Devuelve los datos del pedido.
Listar pedidos	Query	Devuelve una colección de pedidos.

Patron CQRS

- Ventajas:
    - Permite optimizar las lecturas y las escrituras por separado.
    - Facilita escalar los componentes de lectura cuando reciben muchas consultas.
    - Ayuda a organizar sistemas con reglas de negocio complejas.
    - Puede combinarse con Event Sourcing, aunque son patrones distintos.

- Desventajas:
    - Una posible desventaja es que aumenta la complejidad. Para una aplicación pequeña, separar completamente los modelos de lectura y escritura puede ser innecesario.

Ventajas de la mensajería asíncrona: Escalabilidad y Resiliencia

- La **escalabilidad** es la capacidad de un sistema para soportar un aumento de trabajo.
- Imagina que tu tienda recibe 1.000 pedidos en pocos minutos y cada pedido necesita generar una factura.
- Si todo el trabajo se realiza dentro de las peticiones HTTP, la aplicación puede tardar más en responder.
- Con Celery puedes añadir trabajadores que procesen las tareas en paralelo.
- Las principales ventajas son:
    - Repartir el trabajo entre varios procesos.
    - Añadir trabajadores cuando aumenta la demanda.
    - Evitar que las tareas lentas bloqueen la respuesta HTTP.
    - Procesar tareas pendientes de forma gradual cuando existe un pico de actividad.

Ventajas de la mensajería asíncrona: Escalabilidad y Resiliencia

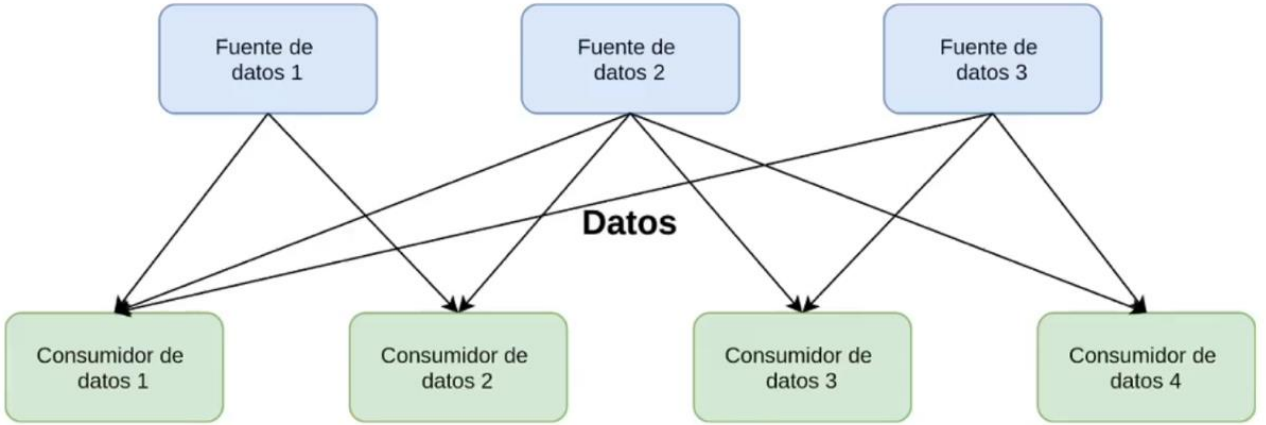
- La **resiliencia** es la capacidad de un sistema para seguir funcionando o recuperarse cuando se producen fallos.
- Por ejemplo, imagina que tu microservicio de pedidos crea correctamente un pedido, pero el servicio de correo está temporalmente caído.
- Si el envío del correo está delegado en una tarea asíncrona, puedes configurar reintentos para intentar enviarlo más adelante sin tener que volver a crear el pedido.
- Las ventajas incluyen:
    - Reintentos: volver a ejecutar tareas que fallan por problemas temporales.
    - Desacoplamiento: un servicio no necesita que todos los demás estén disponibles para completar su propio trabajo.
    - Colas de mensajes pendientes: permiten acumular trabajo cuando los consumidores están ocupados.
    - Recuperación: determinados sistemas de mensajería pueden conservar mensajes y recuperarlos después de un fallo, si están configurados adecuadamente.

Apache Kafka

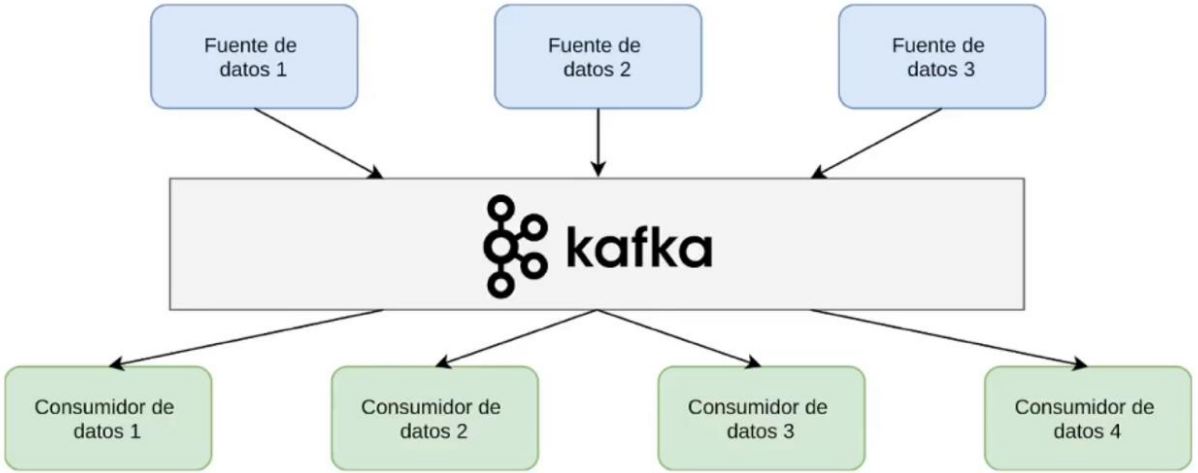
- Apache Kakfa es una plataforma distribuida de transmisión de datos que permite publicar, almacenar y procesar flujos de datos en tiempo real
- Esta escrito en Java y Scala
- La idea inicial surge en Linkedin como solución a un problema interno de desarrollo
- En 2011 Apache se hace cargo del proyecto



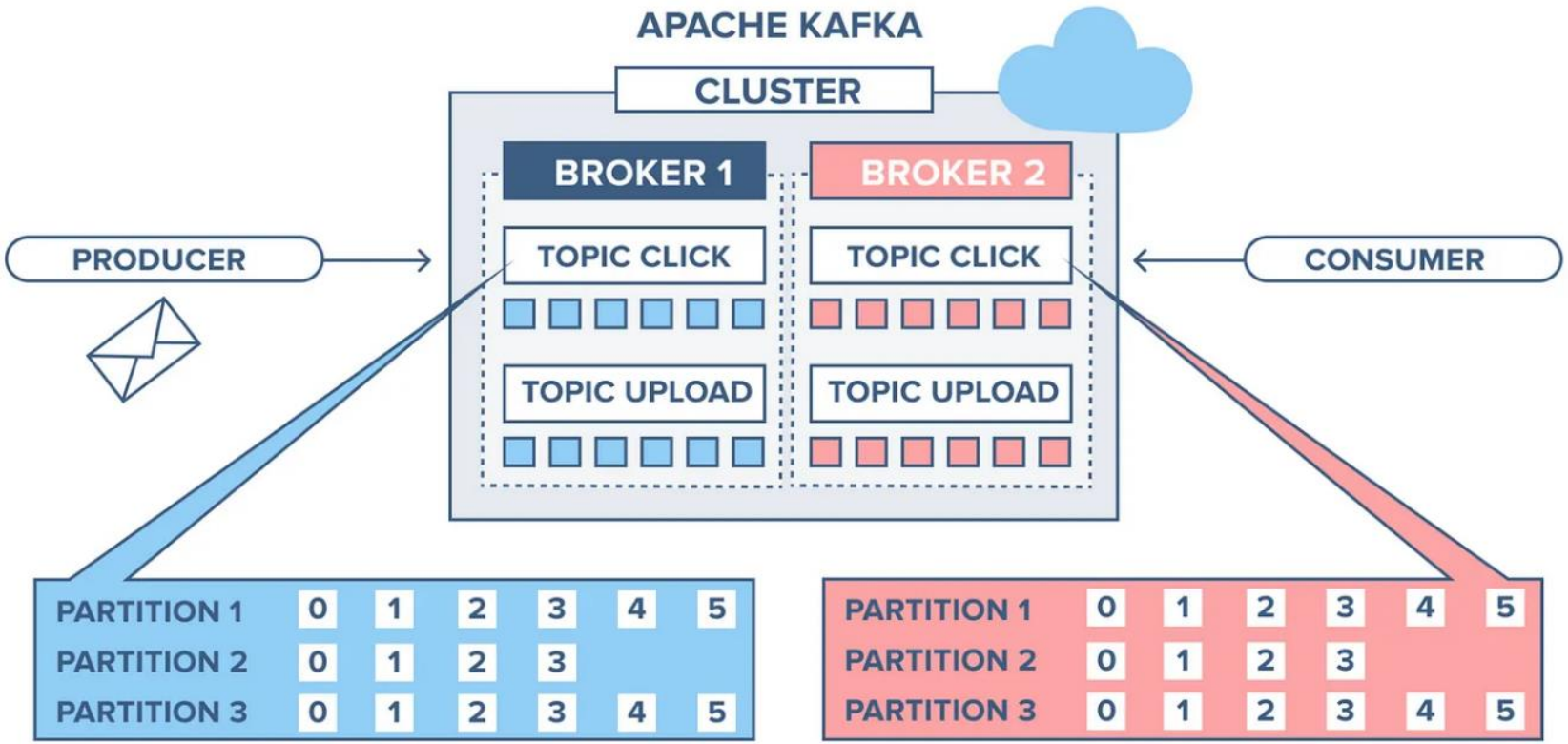
El problema



Apache Kafka



Arquitectura de Apache Kafka



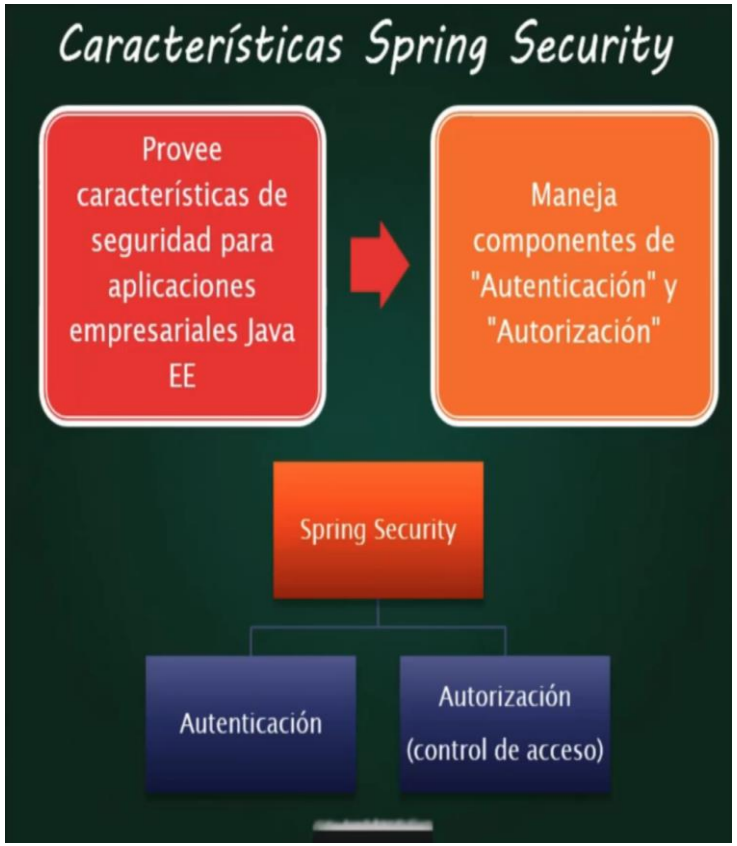
ibertech

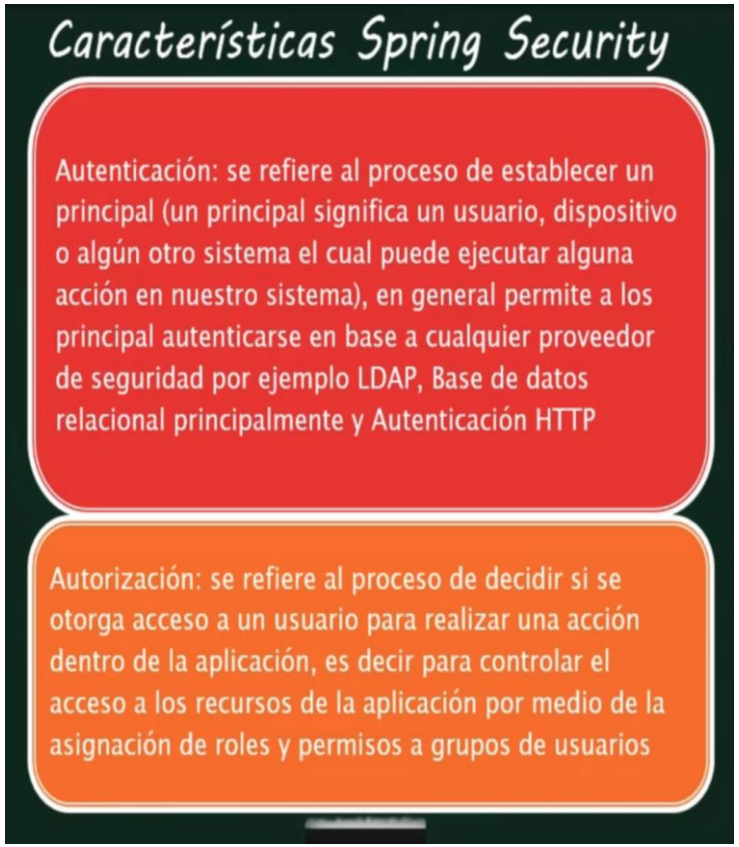
end2end dedication

Tema 4

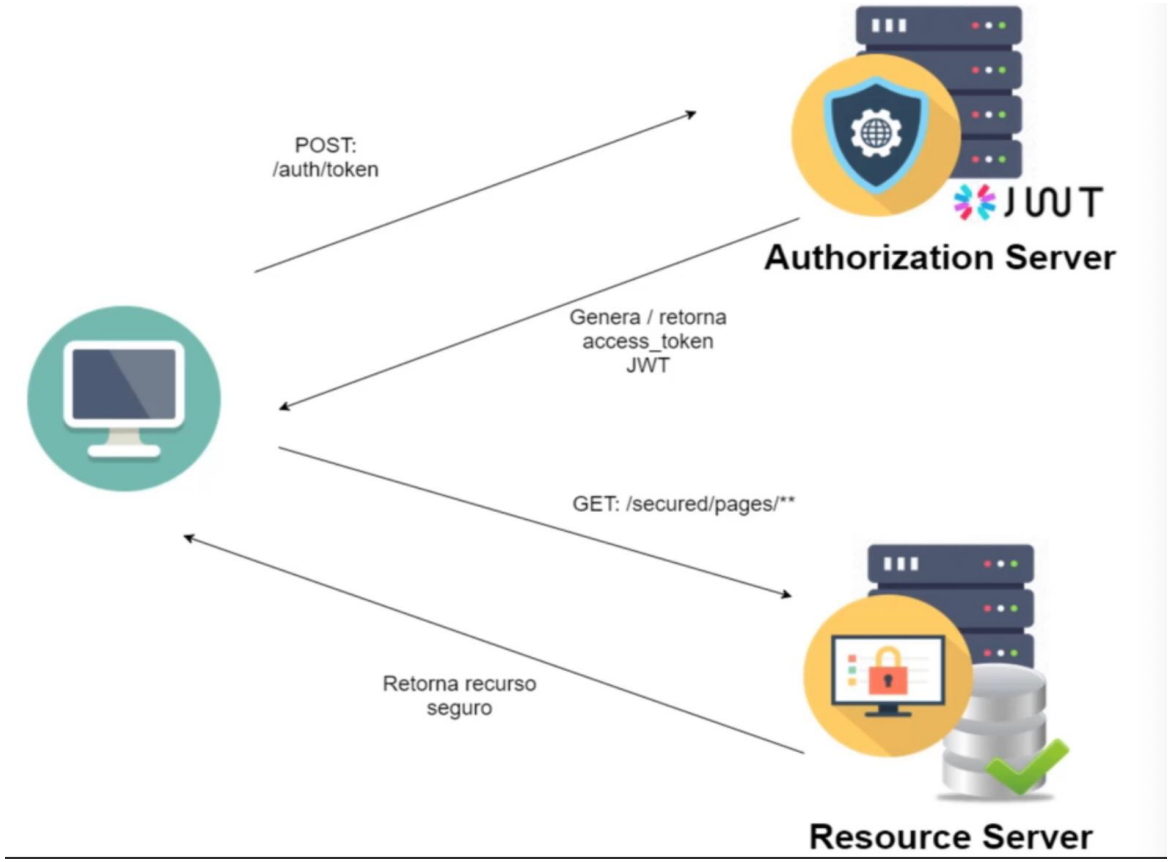
Seguridad y autenticación

Spring Security

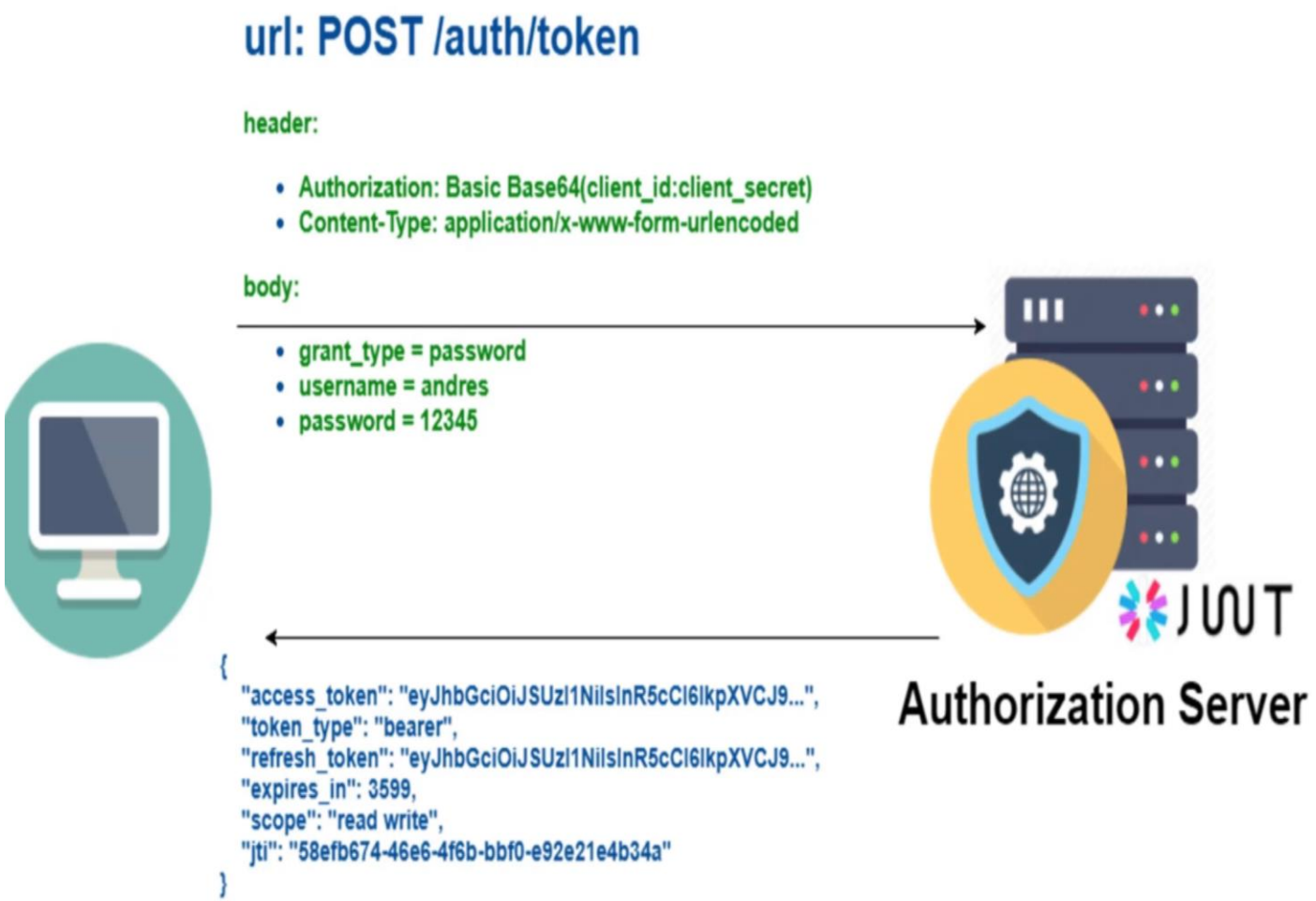




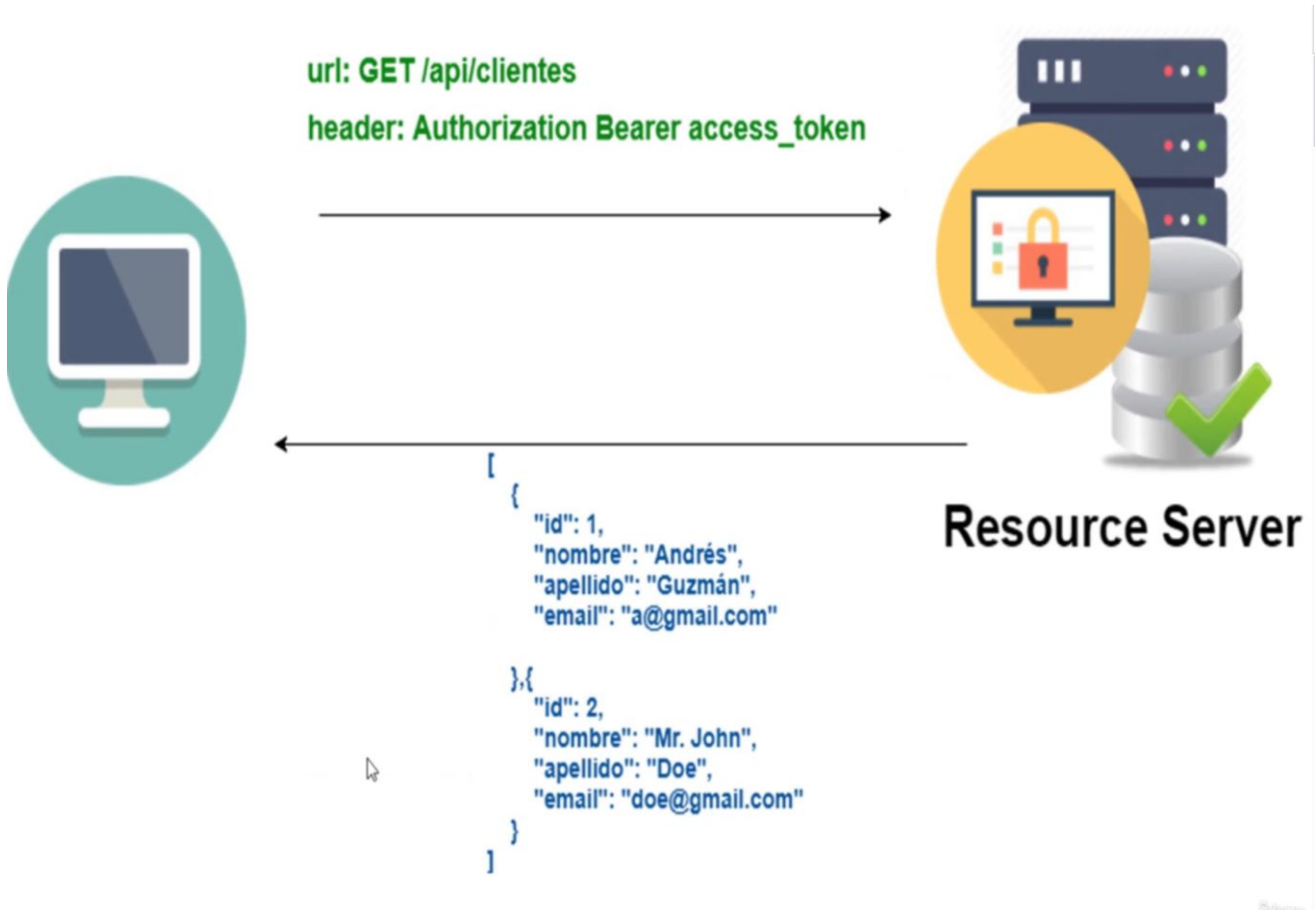
Oauth2


POST:
/auth/token
JWT
Authorization Server
Genera / retorna
access_token
JWT
GET: /secured/pages/**
Retorna recurso
seguro
Resource Server

Authorization Server

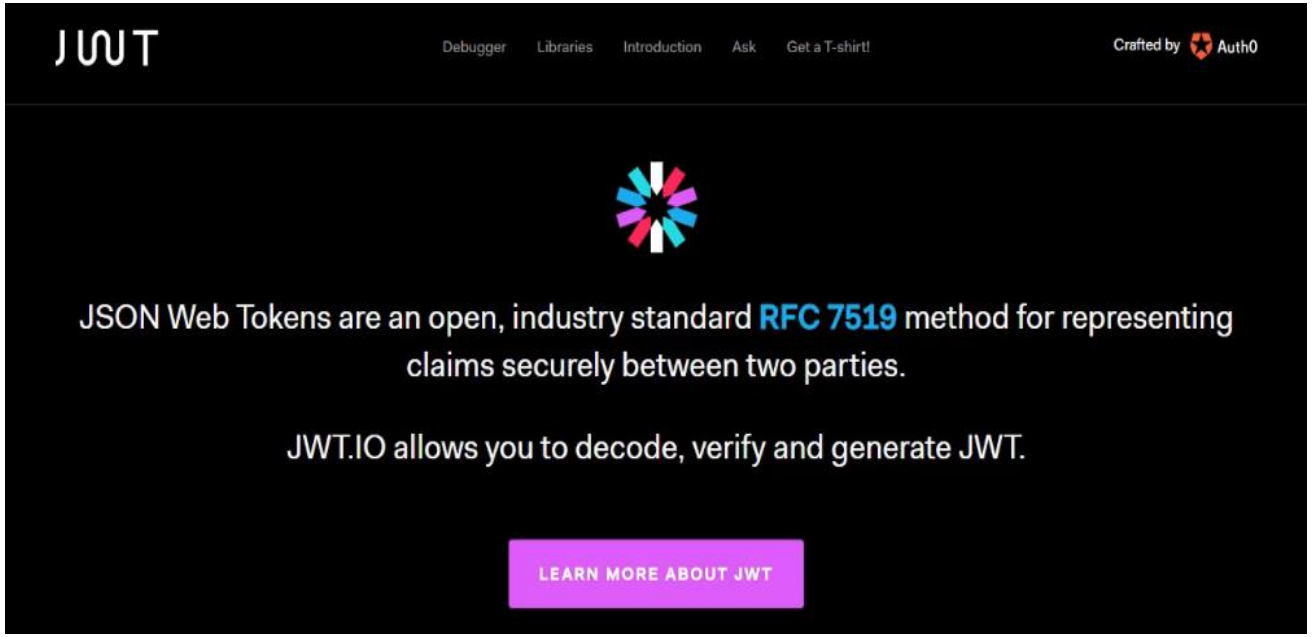


Resource Server

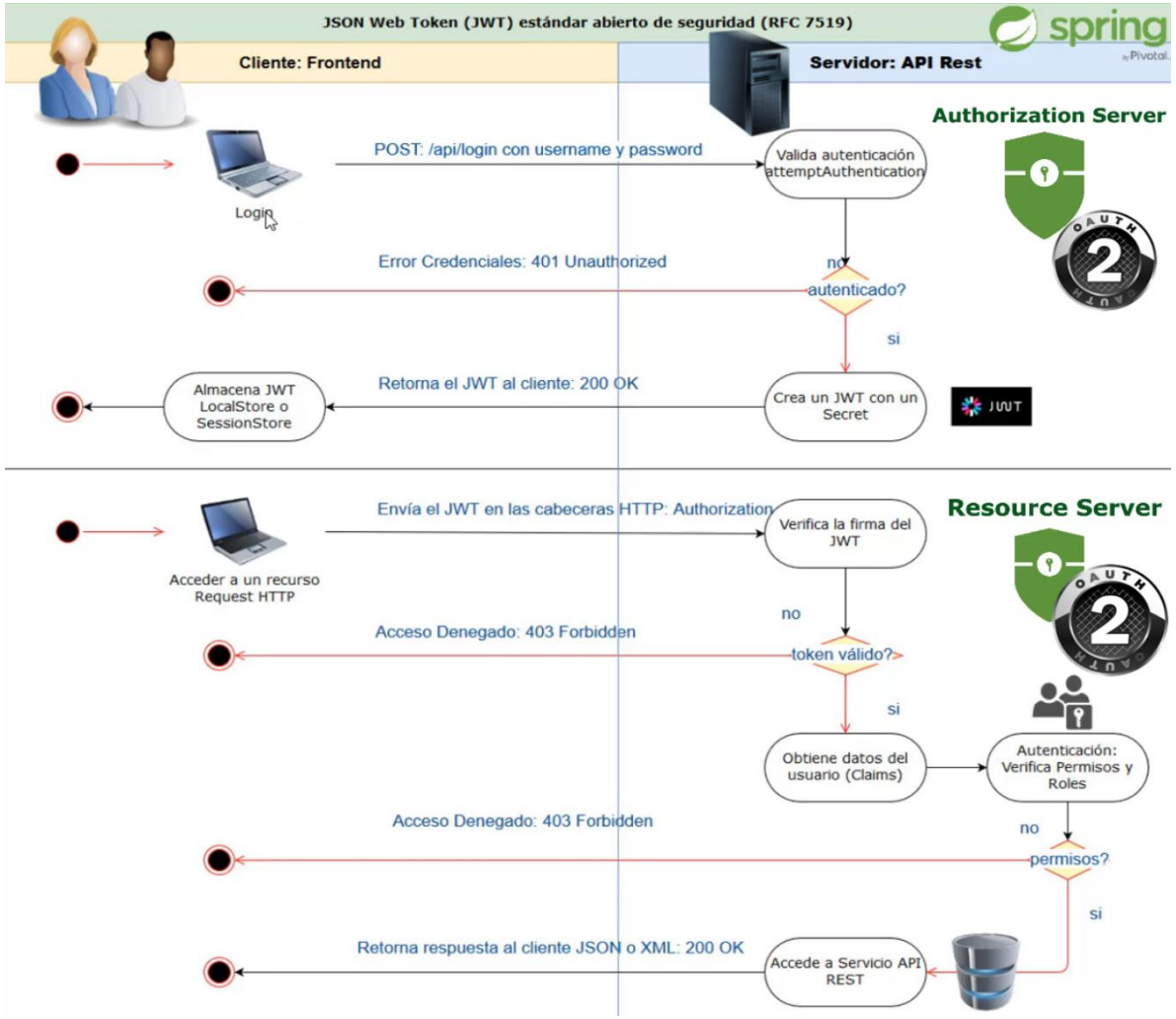


JWT

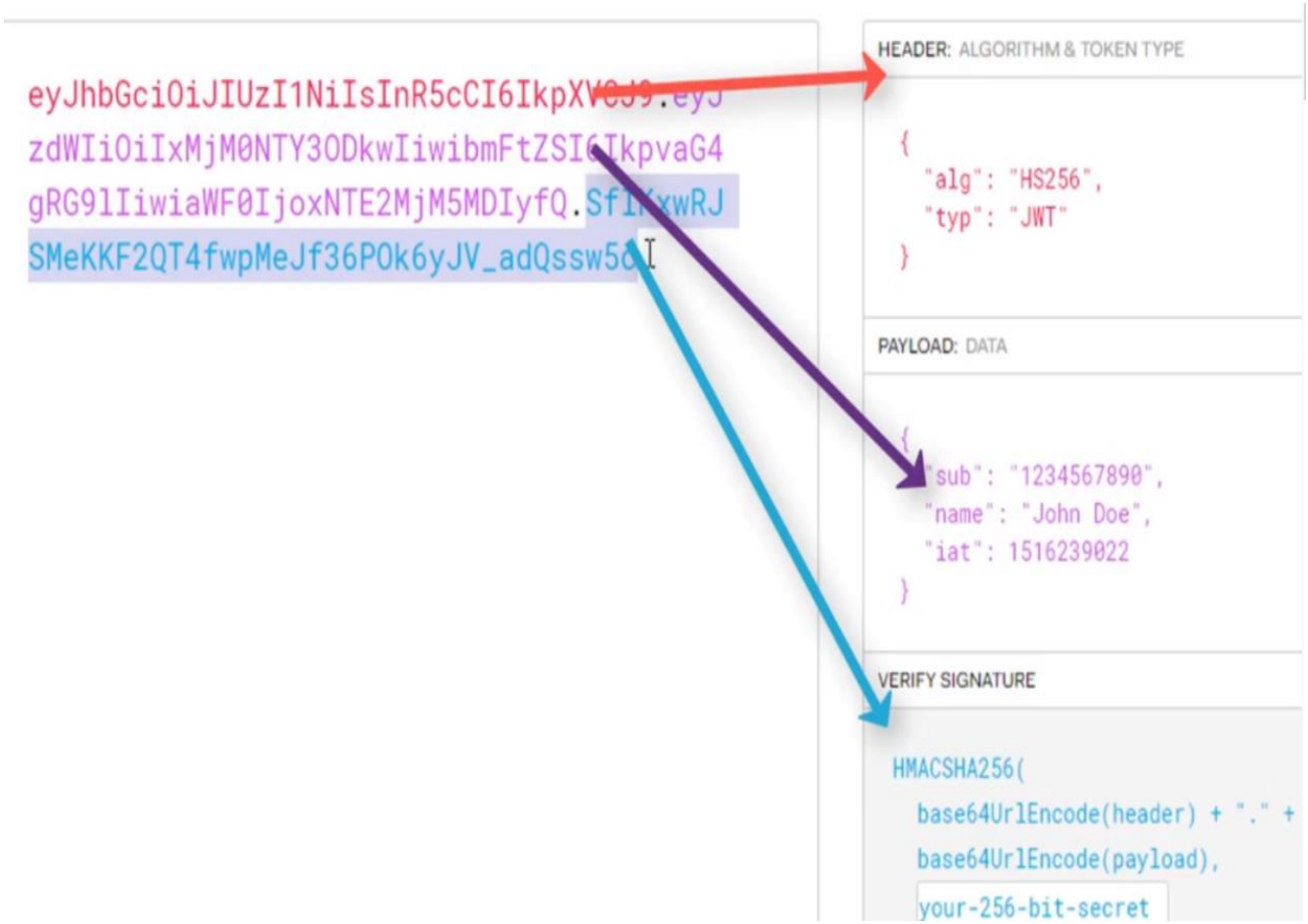
- JWT es un estándar abierto para implementar seguridad en nuestras aplicaciones API Rest.



JWT


JSON Web Token (JWT) estándar abierto de seguridad (RFC 7519)
spring
Cliente: Frontend
Servidor: API Rest
Authorization Server
POST: /api/login con username y password
Valida autenticación attemptAuthentication
Login
Error Credenciales: 401 Unauthorized
no
autenticado?
si
Almacena JWT LocalStore o SessionStore
Retorna el JWT al cliente: 200 OK
Crea un JWT con un Secret
JWT
Resource Server
Envía el JWT en las cabeceras HTTP: Authorization
Verifica la firma del JWT
Acceder a un recurso Request HTTP
no
Acceso Denegado: 403 Forbidden
token válido?
si
Obtiene datos del usuario (Claims)
Autenticación: Verifica Permisos y Roles
Acceso Denegado: 403 Forbidden
no
permisos?
si
Retorna respuesta al cliente JSON o XML: 200 OK
Accede a Servicio API REST

Token


eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXV0J9.eyJ
zdWIiOiIxMjM0NTY3ODkwIiwibmFtZSI6IkpvaG4
gRG9lIiwiaWF0IjoxNTE2MjM5MDIyfQ.SfIKxwRJ
SMeKKF2QT4fwpMeJf36POk6yJV_adQssw5c
HEADER: ALGORITHM & TOKEN TYPE
{
"alg": "HS256",
"typ": "JWT"
}
PAYLOAD: DATA
{
"sub": "1234567890",
"name": "John Doe",
"iat": 1516239022
}
VERIFY SIGNATURE
HMACSHA256(
base64UrlEncode(header) + "." +
base64UrlEncode(payload),
your-256-bit-secret

Autenticación con Google o Facebook

- Para **integrar proveedores externos de autenticación como Google o Facebook en una aplicación Spring Boot**, se utiliza el soporte nativo de **Spring Security OAuth2 Client**. Esto permite que los usuarios inicien sesión usando sus cuentas de Google o Facebook de forma segura y sencilla, sin que la aplicación gestione directamente contraseñas.

- Spring Boot permite integrar autenticación social con Google y Facebook fácilmente configurando los datos de cliente y usando las rutas /oauth2/authorization/google y /oauth2/authorization/facebook.

- Spring Security gestiona automáticamente el flujo OAuth2, la obtención de los datos del usuario y la sesión autenticada, facilitando la implementación de login social seguro y moderno

Implementación de SSL/TLS

- La **implementación de SSL/TLS en microservicios con Spring Boot** es fundamental para proteger la comunicación entre servicios y con clientes externos, asegurando la confidencialidad e integridad de los datos en tránsito. Spring Boot facilita esta configuración tanto para servidores como para clientes.

- Los pasos a seguir son:

    1. Generar o adquirir un certificado SSL/TLS

    2. Configuración del servidor Spring Boot para HTTPS

    3. Proteger endpoints REST

    4. Consumo seguro de APIs entre microservicios

Protección contra ataques comunes

- Spring Boot, junto con Spring Security y buenas prácticas de desarrollo, proporciona mecanismos robustos para **proteger aplicaciones contra ataques comunes como CSRF, XSS y SQL Injection**.
    - CSRF o Cross-Site Request Forgery (Falsificación de Peticiones Cross-Site), es un tipo de ataque cibernético que ocurre cuando un usuario, ya autenticado en un sitio web, es engañado para realizar acciones no deseadas en ese sitio, sin su conocimiento o consentimiento.
    - El cross-site scripting (XSS) es un tipo de vulnerabilidad de seguridad web que permite a los atacantes inyectar código malicioso (generalmente JavaScript) en sitios web legítimos. Este código malicioso se ejecuta en el navegador del usuario, permitiendo al atacante acceder a información sensible, realizar acciones en nombre del usuario o redirigirlo a sitios web maliciosos
    - Las inyecciones de SQL (o SQLI) se producen cuando el hacker introduce o inyecta en el sitio web código SQL malicioso, un tipo de malware que se conoce como la carga útil, y consigue subrepticiamente que envíe ese código a su base de datos como si de una consulta legítima se tratara.

Protección contra CSRF (Cross-Site Request Forgery)

- Spring Security habilita la protección CSRF por defecto en aplicaciones web, generando y validando automáticamente un token CSRF en formularios y peticiones mutables (POST, PUT, DELETE).
- Solo solicitudes legítimas que incluyan el token correcto serán aceptadas por el servidor, bloqueando intentos de ejecución no autorizada desde sitios externos.
- Si tu aplicación expone solo APIs REST (stateless), puedes desactivar CSRF, pero para aplicaciones web tradicionales debe mantenerse activo

Protección contra XSS (Cross-Site Scripting)

- Validación y sanitización de entradas: Siempre valida y limpia los datos recibidos del usuario antes de almacenarlos o mostrarlos.
- Escapado de salidas: Usa frameworks y motores de plantillas (Thymeleaf, JSP, etc.) que escapan automáticamente el contenido mostrado en vistas, evitando la ejecución de scripts maliciosos
- Cabeceras de seguridad: Spring Security añade por defecto cabeceras como X-XSS-Protection y permite configurar políticas CSP (Content Security Policy) para restringir la ejecución de scripts no autorizados

Protección contra SQL Injection

- Uso de consultas preparadas y parámetros: Utiliza siempre consultas preparadas o frameworks ORM como JPA/Hibernate, que separan la lógica SQL de los datos del usuario, evitando la inyección de código malicioso.
- Validación de entradas: Asegúrate de validar y sanear todos los datos que provienen del usuario antes de utilizarlos en consultas a la base de datos.
- Principio de privilegios mínimos: Limita los permisos de la cuenta de base de datos utilizada por la aplicación, restringiendo las acciones que un posible atacante podría realizar en caso de vulnerabilidad.
- Evita concatenar cadenas con datos del usuario en sentencias SQL.

ibertech

end2end dedication

Tema 5

Despliegue de microservicios con Docker y kubernetes

Docker

- Docker es una plataforma de código abierto que permite desarrollar, enviar y ejecutar aplicaciones dentro de contenedores. Estos contenedores son unidades ligeras y portables que incluyen todo lo necesario para ejecutar una aplicación: código, bibliotecas, dependencias y configuración, garantizando que la aplicación funcione de manera idéntica en cualquier entorno que soporte Docker

- Ventajas de Docker:

    - Portabilidad entre diferentes entornos (desarrollo, pruebas, producción)

    - Aislamiento de aplicaciones, evitando conflictos de dependencias

    - Facilidad para escalar y actualizar aplicaciones

    - Eficiencia en el uso de recursos del sistema

Arquitectura de Docker

- Docker sigue una arquitectura cliente-servidor:
  - Docker Daemon (dockerd): Es el proceso principal que gestiona los contenedores y escucha las peticiones de la API de Docker.
  - Cliente de Docker (docker CLI): Es la interfaz de línea de comandos que permite a los usuarios interactuar con el daemon para construir, ejecutar y administrar contenedores.
  - API de Docker: Permite la comunicación entre el cliente y el daemon, facilitando la automatización y la integración con otras herramientas.

Componentes clave de Docker

- **Dockerfile**: Archivo de texto que define las instrucciones para construir una imagen de Docker (por ejemplo, qué sistema base usar, qué dependencias instalar, qué comandos ejecutar).
- **Imagen**: Plantilla de solo lectura creada a partir de un Dockerfile. Contiene el sistema de archivos y las dependencias necesarias para ejecutar una aplicación.
- **Contenedor**: Instancia en ejecución de una imagen. Es el entorno aislado donde se ejecuta la aplicación.
- **Docker Hub**: Repositorio público (o privado) donde se almacenan y comparten imágenes de Docker.

Diferencias de Docker con las Máquinas Virtuales

Docker (Contenedores)	Máquinas Virtuales (VM)
Virtualiza a nivel de sistema operativo	Virtualiza hardware completo
Comparte el kernel del host	Cada VM tiene su propio sistema operativo
Más ligeros y rápidos	Más pesados y lentos
Menor consumo de recursos	Mayor consumo de recursos

Despliegue de microservicios Java en Kubernetes

- El **despliegue de microservicios Java en Kubernetes** es un proceso que implica la creación de imágenes Docker, la definición de recursos de Kubernetes mediante archivos YAML y la gestión de la comunicación, escalabilidad y resiliencia de los servicios. A continuación se describen los pasos y mejores prácticas clave para desplegar aplicaciones Java (por ejemplo, Spring Boot) en un clúster Kubernetes:
    - Empaquetado del microservicio Java
    - Definición de recursos Kubernetes: Deployment, Service, Ingress
    - Despliegue en el clúster

Helm para gestionar el despliegue automatizado

- Helm actúa como gestor de paquetes para Kubernetes, permitiendo empaquetar, versionar, parametrizar y reutilizar despliegues mediante “charts”, facilitando tanto la automatización como la gestión de configuraciones por entorno.
- Ventajas de usar Helm con Spring Boot
  - **Despliegue automatizado y reproducible**: Helm permite instalar, actualizar y revertir aplicaciones de forma sencilla y controlada, asegurando consistencia en todos los entornos
  - **Gestión centralizada de configuración**: Los valores específicos de cada entorno (dev, test, prod) se gestionan en archivos values.yaml, evitando cambios en el código fuente y facilitando la personalización
  - **Versionado y rollback**: Cada despliegue se versiona, permitiendo revertir a versiones anteriores si es necesario, lo que mejora la resiliencia y la capacidad de respuesta ante errores
  - **Reutilización y estandarización**: Los charts pueden compartirse y reutilizarse entre proyectos y equipos, acelerando la adopción de mejores prácticas y la estandarización de despliegues
  - **Soporte de hooks y gestión de ciclo de vida**: Helm permite ejecutar scripts o tareas personalizadas antes o después de cada fase del despliegue (pre-install, post-install, etc.)

Autoescalado de microservicios en Kubernetes con HPA

- El **Horizontal Pod Autoscaler (HPA)** en Kubernetes permite que los microservicios (como los desarrollados con Spring Boot) escalen automáticamente el número de réplicas de sus pods en función de métricas como el uso de CPU, memoria o métricas personalizadas. Esto garantiza que tu aplicación pueda responder eficientemente a picos de demanda y reducir recursos cuando la carga disminuye, optimizando costes y disponibilidad

- Como funciona HPA?
  - **Monitoriza métricas** (por ejemplo, CPU o memoria) de los pods de un Deployment, ReplicaSet o StatefulSet.
  - **Ajusta automáticamente el número de pods** entre un mínimo y un máximo configurado, según los umbrales definidos.
  - **Escalado horizontal**: Si la carga aumenta y supera el umbral, Kubernetes crea más pods; si baja, reduce el número de pods

Balanceo de carga en Kubernetes

- Kubernetes gestiona el balanceo de carga entre servicios mediante la abstracción de Services, distribuyendo el tráfico entre los pods asociados y garantizando disponibilidad, escalabilidad y eficiencia.

- El tipo de Service y el uso de Ingress o herramientas avanzadas permiten adaptar el balanceo a las necesidades específicas de cada aplicación

    - **ClusterIP**: balancea el tráfico entre pods dentro del clúster.

    - **NodePort**: expone el servicio en un puerto específico de cada nodo, permitiendo el acceso externo y el balanceo entre nodos.

    - **LoadBalancer**: solicita un balanceador de carga externo (de un proveedor cloud) que dirige el tráfico a los servicios dentro del clúster.

    - **Ingress**: permite reglas avanzadas de ruteo HTTP/HTTPS, balanceando el tráfico a diferentes servicios según el dominio o la ruta.

ibertech

end2end dedication

Tema 6

Monitorización y Gestión de microservicios con Python

Monitorización del rendimiento de microservicios con Prometheus y Grafana

- La **monitorización del rendimiento de microservicios con Prometheus y Grafana** es una práctica esencial para garantizar la salud, disponibilidad y eficiencia de sistemas distribuidos. Estas herramientas, ampliamente adoptadas en entornos cloud-native y Kubernetes, permiten recopilar, analizar y visualizar métricas en tiempo real, facilitando la detección proactiva de problemas y la optimización continua de los servicios.

- **Prometheus**: es una herramienta de monitorización y alertas que recopila métricas de microservicios mediante peticiones HTTP (modelo pull) y las almacena como series temporales en su base de datos interna. Permite definir alertas y realizar consultas avanzadas usando PromQL.

- **Grafana**: es una plataforma de visualización que se conecta a Prometheus para mostrar las métricas en dashboards interactivos y personalizables. Permite crear paneles para observar el rendimiento, analizar tendencias y recibir alertas visuales

Visualización de métricas en Grafana y configuración de alertas

- Visualización de métricas en Grafana:
  - Grafana es una plataforma líder para la visualización de métricas provenientes de sistemas como Prometheus, InfluxDB y otros.
  - Permite crear dashboards personalizados donde puedes mostrar métricas clave (CPU, memoria, latencia, etc.) en gráficos de líneas, barras, medidores, tablas y más
  - Los paneles se pueden organizar, personalizar (colores, etiquetas, títulos) y adaptar para mostrar información relevante de un vistazo, facilitando el monitoreo en tiempo real y el análisis histórico
  - Puedes ajustar las consultas según la fuente de datos y las necesidades del sistema, y modificar visualmente los parámetros de cada gráfico para mejorar la comprensión y el impacto visual

Visualización de métricas en Grafana y configuración de alertas

- Configuración de alertas en Grafana:
    1. Abre el panel donde visualizas la métrica de interés.
    2. Define una consulta (por ejemplo, uso de CPU, latencia, etc.).
    3. Haz clic en la opción para añadir alerta y establece:
        - Nombre de la alerta.
        - Frecuencia de evaluación (por ejemplo, cada 1 minuto).
        - Condición (por ejemplo, si el valor medio de CPU supera el 90% durante 5 minutos)
    4. Configura el canal de notificación: Grafana soporta correo electrónico, Slack, Teams, Telegram y otros
    5. Guarda la alerta y, si lo deseas, realiza un test para verificar su funcionamiento

Implementación de tracing distribuido con Spring Cloud Sleuth y Jaeger

- El tracing distribuido es fundamental para observar, depurar y analizar el flujo de peticiones en arquitecturas de microservicios. Spring Cloud Sleuth y Jaeger permiten instrumentar aplicaciones Spring Boot para obtener visibilidad completa de las transacciones distribuidas.
- **Spring Cloud Sleuth** es una librería que añade identificadores de traza (**traceId**) y de operación (**spanId**) a las peticiones que atraviesan los microservicios. Estos identificadores se propagan automáticamente entre servicios, permitiendo correlacionar logs y reconstruir el recorrido completo de una petición
- **Jaeger** es una plataforma open source para tracing distribuido desarrollada por Uber. Permite recolectar, almacenar y visualizar trazas generadas por los microservicios, facilitando el análisis de latencias, dependencias y cuellos de botella en sistemas distribuidos
- Arquitectura típica:
    - Microservicios instrumentados con Sleuth: Cada servicio añade automáticamente los identificadores de traza y propaga los contextos.
    - Jaeger Agent/Collector: Recibe las trazas y las almacena.
    - Jaeger UI: Permite consultar y analizar visualmente las trazas completas.

Gestión de logs centralizados con Elasticsearch, Fluentd y Kibana

- La gestión centralizada de logs es fundamental en arquitecturas modernas, especialmente en entornos de microservicios y Kubernetes, donde múltiples aplicaciones y servicios generan grandes volúmenes de registros. El stack EFK (Elasticsearch, Fluentd y Kibana) es una de las soluciones más populares y robustas para recolectar, almacenar, analizar y visualizar logs de manera centralizada.

- Que es el stack EFK?

  - **Elasticsearch**: Motor de búsqueda y análisis distribuido, encargado de almacenar e indexar los logs para búsquedas y análisis rápidos.

  - **Fluentd**: Agente de recolección de logs que recopila, transforma y reenvía los registros desde diversas fuentes hacia Elasticsearch.

  - **Kibana**: Interfaz web para visualizar, explorar y crear dashboards a partir de los datos almacenados en Elasticsearch

Gestión de logs centralizados con Elasticsearch, Fluentd y Kibana

- Arquitectura y flujo de datos:
  - **Recolección**: Fluentd se despliega en cada nodo (por ejemplo, como DaemonSet en Kubernetes) y recopila logs de aplicaciones, contenedores y del sistema operativo.
  - **Transformación y envío**: Fluentd puede filtrar, transformar y enriquecer los logs antes de enviarlos a Elasticsearch.
  - **Almacenamiento e indexación**: Elasticsearch almacena los logs en índices, permitiendo búsquedas complejas y agrupaciones.
  - **Visualización**: Kibana se conecta a Elasticsearch y permite consultar, visualizar y crear alertas o dashboards personalizados sobre los logs centralizados

Gestión de logs centralizados con Elasticsearch, Fluentd y Kibana

- Ventajas del stack EFK:
  - **Centralización**: Todos los logs se almacenan en un solo lugar, facilitando la gestión, auditoría y análisis.
  - **Escalabilidad**: Elasticsearch permite almacenar y consultar grandes volúmenes de datos y escalar horizontalmente.
  - **Visualización avanzada**: Kibana proporciona potentes herramientas de visualización y análisis en tiempo real.
  - **Flexibilidad**: Fluentd puede adaptarse a múltiples fuentes y formatos de logs, integrándose fácilmente con diferentes aplicaciones y plataformas.
  - **Trazabilidad y troubleshooting**: Permite rastrear eventos y errores a través de múltiples servicios, facilitando la detección y resolución de problemas

ibertech

end2end dedication

Muchas gracias por vuestra asistencia

www.ibertech.es